

Verifica automatica di programmi

Riccardo Torre

17 aprile 2026

1 BubbleSort

Definizione di Partitioned

$$\overbrace{\text{Partitioned}(a, l_1, u_1, l_2, u_2)}^P \equiv \forall i \forall j: l_1 \leq i \leq u_1 < l_2 \leq j \leq u_2 \implies a[i] \leq a[j]$$

Definizione di Sorted

$$\overbrace{\text{Sorted}(a, l, u)}^S \equiv \forall i \forall j: l \leq i \leq j \leq u \implies a[i] \leq a[j]$$

```
1  pre:   $\top$ 
2  post: sorted(rv, 0, |rv| - 1)
3
4  int[] BubbleSort(int[] a0){
5
6      for @L1:  $-1 \leq i < |a| \wedge \overbrace{P(a, 0, i, i+1, |a|-1)}^A \wedge \overbrace{S(a, i, |a|-1)}^B$ 
7      (int i := |a| - 1; i > 0; i := i+1){
8          for @L2:  $1 < i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j-1, j, j)$ 
9          (int j := 0; j < i; j := j+1)
10             if(a[j] > a[j+1]){
11                 int t := a[j];
12                 a[j] := a[j+1];
13                 a[j+1] := t;
14             }
15     }
16     return a;
}
```

1.1 Cammini elementari di BubbleSort

Inizio ciclo esterno $P(a, \overbrace{0}^{l_1}, \overbrace{|a|-1}^{u_1}, \overbrace{|a|-1}^{l_2}, \overbrace{|a|-1}^{u_2})$ è vera a vuoto perché $\nexists j$ che verifica. Applichiamo la definizione di **Partitioned**.

$$\overbrace{\text{Partitioned}(a, l_1, u_1, l_2, u_2)}^P \equiv \forall i \forall j: l_1 \leq i \leq u_1 < l_2 \leq j \leq u_2 \implies a[i] \leq a[j]$$

Sostituendo si ottiene $P(a_0, 0, |a_0| - 1, |a_0|, |a_0| - 1) \equiv \forall i \forall j: 0 \leq i \leq |a_0| - 1 < |a_0| \leq j \leq |a_0| - 1 \implies a[i] \leq a[j]$. Ma $\nexists j: |a_0| \leq j \leq |a_0| - 1$ perché $|a_0| - 1 \not\leq |a_0|$.

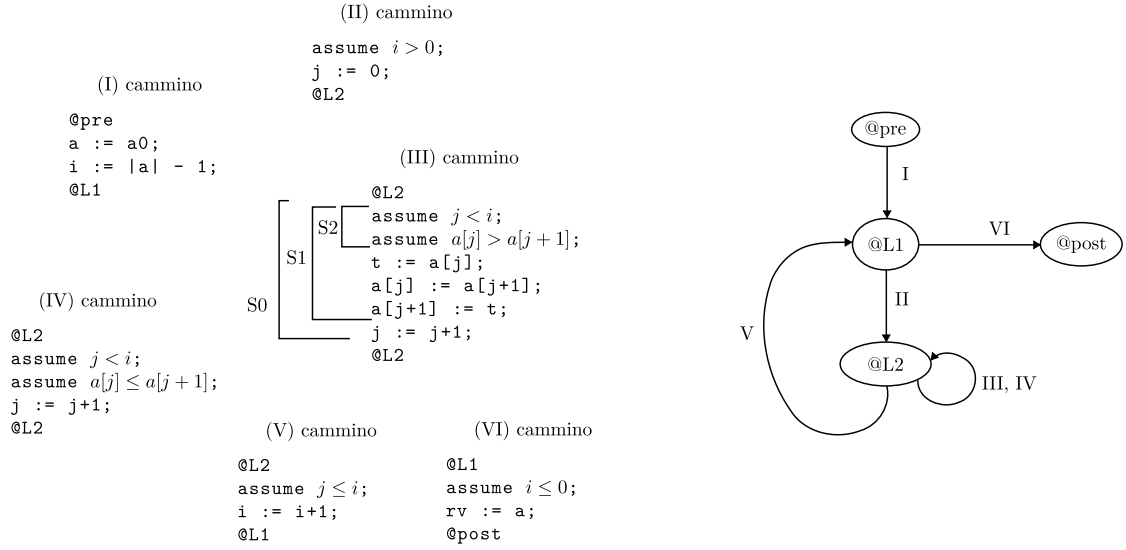


Figura 1: cammini minimi e grafo di BubbleSort

Inizio ciclo interno $P(a, 0, -1, 0, 0)$ è vera a vuoto poiché tra 0 e -1 non esiste nulla ($\nexists i \in [0, -1]$).

1.2 Condizioni di verifica per i cammini del grafo

Weakest precondition del cammino (I) $wp(-1 \leq i < |a| \wedge A \wedge B, a := a0; i := |a| - 1;) \equiv (-1 \leq |a| - 1 < |a| \wedge P(a, 0, |a| - 1, |a|, |a| - 1) \wedge S(a, |a| - 1, |a| - 1), a := a0) = (-1 \leq |a_0| - 1 < |a_0| \wedge P(a_0, 0, |a_0| - 1, |a_0|, |a_0| - 1) \wedge S(a_0, |a_0| - 1, |a_0| - 1))$.

Condizione di verifica per il cammino (I) $T \implies (-1 \leq |a_0| - 1 < |a_0| \wedge P(a_0, 0, |a_0| - 1, |a_0|, |a_0| - 1) \wedge S(a_0, |a_0| - 1, |a_0| - 1))$ è vera perché:

- P è vera ✓
- S è vera ✓
- $-1 \leq |a_0| - 1 < |a_0|$ vera anche per $|a_0| = 0$ ✓

Weakest precondition del cammino (II) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j - 1, j, j), \text{assume } i > 0; j := 0;) \equiv i > 0 \implies 1 \leq i < |a| \wedge 0 \leq 0 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, -1, 0, 0)$.

Condizione di verifica per il cammino (II) $[-1 \leq i < |a| \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge i > 0] \implies [1 \leq i < |a| \wedge 0 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, -1, 0, 0)]$. L'implicazione è vera.

Weakest precondition del cammino (III) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, j - 1, j, j), S_0) = wp(1 \leq i \leq |a| \wedge 0 \leq j + 1 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, j, j + 1, j + 1), S_1) = wp(1 \leq i < |b| \wedge 0 \leq j + 1 \leq i \wedge P(b, 0, i, i + 1, |b| - 1) \wedge S(b, i, |b| - 1) \wedge P(b, 0, j, j + 1, j + 1), S_2)$

$t := a[j]$ diventa $t \leftarrow \text{select}(a, j)$
 $a[j] := a[j+1]$; diventa $a \leftarrow \text{store}(a, j, \text{select}(a, j+1))$
 $a[j+1] := t$; diventa $a \leftarrow \text{store}(a, j+1, t)$
 combinando la seconda e la terza formula si ottiene $a \leftarrow \text{store}(\text{store}(a, j, \text{select}(a, j+1)), j+1, t)$ e sostituendo t con la prima formula si ottiene $a \leftarrow \text{store}(\text{store}(a, j, \text{select}(a, j+1)), j+1, \text{select}(a, j)) \equiv b$ dove b è una **variabile temporanea** di tipo array.

dunque la weakest precondition per il cammino (III) risulta: $[j < i \wedge \text{select}(a, j) > \text{select}(a, j+1)] \implies [1 \leq i < |b| \wedge 0 \leq j+1 \leq i \wedge P(b, 0, i, i+1, |b|-1) \wedge S(b, i, |b|-1) \wedge P(b, 0, j, j+1, j+1)]$

Condizione di verifica per il cammino (III) $[1 \leq i < |a| \wedge 0 \leq j \leq i \wedge P(a, 0, i, i+1, |a|-1) \wedge S(a, i, |a|-1) \wedge P(a, 0, j-1, j, j) \wedge j < i \wedge \text{select}(a, j) > \text{select}(a, j+1)] \implies [1 \leq i < |b| \wedge 0 \leq j+1 \leq i \wedge P(b, 0, i, i+1, |b|-1) \wedge S(b, i, |b|-1) \wedge P(b, 0, j, j+1, j+1)]$.

L'implicazione è vera siccome b è il risultato di due **store** e allora $|b| = |a|$ ed è facile vedere che vengono le altre parti dell'implicazione.

Weakest precondition del cammino (IV) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j-1, j, j, \text{assume } j < i; \text{assume } a[j] \leq a[j+1]; j := j+1;)) \equiv [j < i \wedge \text{select}(a, j) \leq \text{select}(a, j+1)] \implies [1 \leq i < |a| \wedge 0 \leq j+1 \leq i \wedge A \wedge B \wedge P(a, 0, j, j+1, j+1, j+1)]$.

Condizione di verifica per il cammino (IV) $[1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j-1, j, j) \wedge j < i \wedge \text{select}(a, j) \leq \text{select}(a, j+1)] \implies [1 \leq i < |a| \wedge 0 \leq j+1 \leq i \wedge A \wedge B \wedge P(a, 0, j, j+1, j+1)]$. L'implicazione è vera.

Weakest precondition del cammino (V) $wp(1 \leq i < |a| \wedge P(a, 0, i, i+1, |a|-1) \wedge S(a, i, |a|-1), \text{assume } j \geq i; i := i-1;) \equiv [j \geq i] \implies [-1 \leq i-1 < |a| \wedge P(a, 0, i-1, i, |a|-1) \wedge S(a, i-1, |a|-1)]$.

Condizione di verifica per il cammino (V) $[1 \leq i < |a| \wedge 0 \leq j \leq i \wedge P(a, 0, i, i+1, |a|-1) \wedge S(a, i, |a|-1) \wedge P(a, 0, j-1, j, j) \wedge j \geq i] \implies [-1 \leq i-1 < |a| \wedge P(a, 0, i-1, i, |a|-1) \wedge S(a, i-1, |a|-1)]$. L'implicazione è vera infatti:

- $1 \leq i \implies 0 \leq i-1 \implies -1 \leq i-1 \longrightarrow \text{ok}; i < |a| \implies i-1 < |a|$ ✓
- $j \leq i \wedge j \geq i \implies j = i$. Si sostituisce j con i :
 - $P(a, 0, i-1, i) \wedge P(a, 0, i, i+1, |a|-1)$ ✓
 - $S(a, i-1, |a|-1) \wedge P(a, 0, i-1, i, i) \implies S(a, i-1, |a|-1)$ ✓

Weakest precondition del cammino (VI) $wp(S(rv, 0, |rv|-1), \text{assume } i \leq 0; rv := e;) \equiv [i \geq 0] \implies [S(a, 0, |a|-1)]$

Condizione di verifica per il cammino (VI) $[-1 \leq i < |a| \wedge P(a, 0, i, i+1, |a|-1) \wedge S(a, i, |a|-1) \wedge i \leq 0] \implies S(a, 0, |a|-1)$. L'implicazione è vera perché $-1 \leq i \wedge i \leq 0 \implies i = -1 \vee i = 0$

- Se $i = -1$ allora **a** era vuoto ($|a| = 0$). Allora $S(a, -1, |a|-1)$ vero a vuoto ✓
- Se $i = 0$ allora **a** aveva un solo elemento $S(a, -1, |a|-1) \implies S(a, 0, |a|-1)$ ✓

2 BinarySearch

Ricorsione Correttezza parziale di funzioni ricorsive via generazione delle chiamate di funzione sui cammini elementari:

- Asserzioni in chiamata;
- Sommario di chiamata:

Listing 1: profilo della funzione f

```

1  @pre: F[p1,...,pn] // sono i parametri formali di una funzione f
2  @post: G[P1, ... Pn, rv] // con rv valore restituito
3  tipo f(type p1, ... type pn)
4  //tipo  $\omega := f(e1, \dots, en); \rightarrow$  chiamata di f

```

- Occorre almeno un cammino che termini con **@R**: $F[e1, \dots, en]$ applicando la sostituzione di $p1, \dots, pn$ con $e1, \dots, en$.
- Dimostrare la validità della condizione di verifica associata a ogni cammino che termina. L'asserzione di chiamata serve a dimostrare che la chiamata della funzione soddisfa la preconditione delle funzione.
- Occorre almeno un cammino che includa il sommario di chiamata **assume** $G[e1, \dots, en, v]$; $\omega := v$; dove v è il valore restituito dalla funzione chiamata ed $e1, \dots, en$ sostituiscono $p1, \dots, pn$.

Nell'analisi di funzioni ricorsive, i cammini elementari iniziano con la preconditione e finiscono con la postcondizione o con un'altra asserzione.

2.1 Esempio

```

1  @pre: x >= 0
2  @post: rv >= x
3  int g(int x){...}

```

Considerando una funzione che chiama g

$$\omega := g(n+1)$$

n è definita nella funzione che chiama g . Bisogna annotare la chiamata di g :

```

1  @R1: n + 1 >= 0 //  $\rightarrow$  asserzione in chiamata
2   $\omega := g(n+1);$ 

```

1. Almeno un cammino che termina su **@R**
2. Almeno un cammino che includa: ... **assume** $v \geq n+1$; $\omega := v$; con v valore gestito da g .